

0–k BFS – Текст доклада

Сопровождение к `presentation.html`

Артёмов Макар, Марк Шкут
Сколтех, Алгоритмы 2026

21 мая 2026

Буквальный текст для каждого слайда из `presentation.html`. Один раздел здесь = один слайд. Зачитываем абзац, и при пунктирной линии переходим к следующему слайду.

1. Титул – “0–k BFS”

“Добрый день. Меня зовут Макар, со мной Марк. Мы построили реализацию на C++17 семейства алгоритмов 0–k BFS для кратчайших путей и сравнили её с Дейкстрой – это наш результат по Этапу 1. Доклад минут на пятнадцать, вопросы в конце.”

2. Постановка задачи

“Задача – кратчайшие пути из одной вершины во взвешенном графе, где веса рёбер – неотрицательные целые числа из $\{0, 1, \dots, k\}$, и k – небольшая известная константа, обычно четыре, восемь или шестнадцать. Для произвольных весов классический ответ – Дейкстра за $O((V + E) \log V)$. Идея доклада в том, что когда диапазон весов мал и известен, мы можем убрать логарифмический фактор. Цель Этапа 1 – построить эту реализацию, сравнить её с базовой Дейкстрой и довести размер входа до 10^9 вершин, насколько позволяет одна обычная рабочая станция. Мотивирующие приложения: дорожные сети с квантованным временем поездки, поиск пути на сетках в играх и робототехнике, и семейство BFS, стандартное в олимпиадном программировании.”

3. Заголовочный результат – сетка 200×200

“Сначала суть на маленьком примере. Одна и та же сетка 200×200 , $k = 8$, один и тот же источник. Оба алгоритма выдают одно и то же поле расстояний – цветовая картина идентична. Числа в заголовках панелей – реально замеренное время: 0–k BFS заканчивает за четыре миллисекунды, Дейкстра – за тринадцать с третью. То есть Дейкстра тратит в 3.32 раза больше времени, чтобы прийти к тому же ответу.”

4. Большое число – $V = 9 \cdot 10^8$

“Теперь заголовок на масштабе. Берём неявную сетку тридцать тысяч на тридцать тысяч – 900 миллионов вершин, 3.6 миллиарда рёбер – и запускаем кратчайшие пути из одной вершины в один поток на машине с 32 гигабайтами оперативной памяти. Дейкстра: 619 секунд. 0–1 BFS на деке: 121 секунда. В 5.12 раза быстрее. Контрольная сумма расстояний совпадает побайтово, корректность подтверждена. Два трюка, благодаря которым это вообще влезает в обычную память: неявная сетка – рёбра не храним, соседей вычисляем за $O(1)$ – и 32-битный тип расстояния, что безопасно, потому что $k \cdot V$ помещается в 2^{32} .”

5. Три алгоритма

“Реализовали три алгоритма за одной абстракцией. Базовый, `dijkstra_pq`, – стандартная `std::priority_queue` с ленивым `decrease-key`. Затем `bfs_01` – 0–1 BFS на `std::deque`. И `bfs_0k` – $k+1$ кольцевая FIFO-очередь для общего диапазона весов. Все три шаблонизированы над адаптером графа, поэтому один и тот же код алгоритма работает и на неявной `GridGraph` – которая масштабируется до

$V \approx 10^9$ – и на классическом **GraphCSR**, который из-за хранения рёбер доходит до $V \approx 10^8$.” —

6. 0–1 BFS: дек

“0–1 BFS – простейший случай в семействе. При весах 0 или 1 достаточно одного дека. Достаём с фронта, релаксируем каждого соседа: если вес ребра 0, кладём новую вершину на фронт – она в том же слое, что и текущая; если вес 1 – в конец, следующий слой. Инвариант: в любой момент в деке максимум два соседних слоя расстояний. Логарифмический множитель Дейкстры пропадает: все операции амортизировано $O(1)$. Проверка `dist[u] < cur_dist_of(u)` пропускает устаревшие записи – одна вершина может быть открыта несколько раз до того, как её расстояние станет финальным.” —

7. 0–k BFS: кольцевые FIFO

“Обобщаем с весов $\{0, 1\}$ на $\{0, 1, \dots, k\}$. Ключевое наблюдение: релаксация двигает вершину не дальше чем на k шагов вперёд по расстояниям. Значит, в любой момент живых слоёв расстояний не больше $k+1$. Заводим $k+1$ FIFO-очередей с индексом по расстоянию по модулю $k+1$ – кольцевой буфер бакетов. Указатель `cur` движется через них вперёд; когда активный бакет пустеет, `cur` сдвигается, и следующий бакет становится активным. Релаксация кладёт цель в бакет (`new_dist`) $\bmod M$, иногда не в тот же, где была старая запись – старая запись просто становится устаревшей и пропускается при поп. `cur` движется только вперёд – поэтому буфер кольцевой и куча не нужна.” —

8. Механика очередей, пошагово

“Это педагогическая визуализация структуры данных. Маленький пример: сетка 10×10 , $k = 3$. Справа – четыре реальные очереди $Q[0] - Q[3]$ с их содержимым; числа в клетках – идентификаторы вершин. Слева сетка, каждая клетка раскрашена по бакету, в котором сейчас лежит её вершина. Красная стрелка `cur` – активный бакет. Видно, как релаксация кладёт вершину в бакет, выбранный (новым расстоянием) $\bmod$ четыре; как активный бакет вычерпывается в FIFO-порядке; как при опустошении `cur` сдвигается; и как устаревшие записи от прошлых релаксаций просто пропускаются при поп. Это весь алгоритм.” —

9. Волна 0–k BFS – 120×120 , $k=4$

“На большей сетке с $k = 4$ и источником у центра фронт волны выглядит как концентрические кольца расстояний. Та же форма, что и у фронта Дейкстры – но без логарифмических накладных расходов на каждый поп. Узор градиента задан случайным рельефом, поэтому кольца не идеально круглые.” —

10. Датасеты

“Бенчмарки используют четыре синтетических генератора, всё через сидированный `std:mt19937_64`, так что каждый запуск воспроизводим. Неявная сетка – наш основной бэкенд – даёт каждой вершине байт “рельефа” и выводит веса рёбер из соседних рельефов, получая структурированные “горы и долины”. Дальше Erdős–Rényi $G(n, p)$ с весами равномерно из $\{0..k\}$, слоистый DAG из L слоёв ширины W с заданным `fanout`, и связательный chain-граф – цепочка со случайными хордами. На сеточном бэкенде рёбра вычисляются на лету – именно этот приём позволяет V дойти до 10^9 без докупки памяти.” —

11. Визуализация поля расстояний

“Два статических примера выхода алгоритма. Слева – сетка 80×80 с источником в углу; на heatmap видно, как поле расстояний формируется случайным рельефом. Справа – граф Erdős–Rényi, нарисованный по кругу, цвет вершины кодирует расстояние от источника. Се-

рые вершины и там, и там – недостижимые. Это вид для проверки: до того, как запускать бенчмарки, мы смотрели на эти картинки, чтобы убедиться, что алгоритмы дают одинаковую картинку пиксель в пиксель.”

12. Масштабирование по V

“Два графика, одни и те же данные, обе оси логарифмические. Слева – время работы от V . Дейкстра наверху, варианты BFS идут примерно параллельно ниже. Постоянный вертикальный зазор в логарифмической шкале – это постоянное мультипликативное ускорение. Справа – само ускорение. Deque-вариант 0–1 BFS пикует около 11 раз быстрее Дейкстры при $V = 10^8$, затем спадает по мере перехода в memory-bound режим. Остальные варианты BFS кучкуются в районе трёх-пяти раз.”

13. Таблица результатов (сетка, 1 поток)

“Проходим таблицу сверху вниз. На миллионе вершин – в 4.6 раза быстрее. Десять миллионов – в 5.6. Сто миллионов при $k = 1$, deque-вариант 0–1 BFS даёт 11.1 раза – это пик. Четвёртая строка интересна: на полумиллиарде вершин оба алгоритма становятся memory-bandwidth bound, и разрыв схлопывается до 0.96 – по сути ничья. Нижняя строка, наш заголовочный результат: на 900 миллионах вершин 0–1 BFS за 121 секунду, Дейкстра – 620. Каждый массив расстояний в свипе кросс-валидирован с Дейкстрой – ноль расхождений.”

14. Где это ломается: k и память

“Честно про границы. Во-первых, кроссовер по k . На миллионе вершин: $k = 1$ даёт 4.6 раза, $k = 4 - 4.1$, $k = 16 - 3.3$. Как только k сравним с $\log V$, асимптотическое преимущество исчезает. Во-вторых, потолок по пропускной способности памяти при $V \gtrsim 5 \cdot 10^8$: оба алгоритма упрутся в память, пиковые push доходят до 600 миллионов, и cache-aware Дейкстра догоняет. В-третьих, дек против очередей: при $V = 10^8$ и $k = 1$ deque-вариант – 5.4 секунды, двухочередной – 28. Та же сложность; вся разница – накладные расходы аллокатора и кэша от двух `std::queue` против одной `std::deque`.”

15. Структура репозитория

“Краткий обзор. `include/zkbfs/` – header-only алгоритмы и типы графов: `common`, `graph_csr`, `grid_graph`, `dijkstra`, `bfs01`, `bfs0k`. `src/` – три CLI-исполняемых файла: `main_bench` для бенчмарков, `main_dump` для визуализаций, `main_trace` для пошагового запуска с инструментацией. `viz/` – Python: драйвер свипа, графики, GIF-анимации. Сборка: обычный `make` или `cmake -S . -B build && cmake -build build`.”

16. Дальнейшие шаги (Неделя 2)

“Четыре пункта в плане. Первый: индексная 4-арная куча для Дейкстры с настоящим `decrease_key` – более честный baseline. Второй: кусочный кольцевой FIFO для `bfs_0k`, чтобы закрыть кэш-разрыв с deque-вариантом 0–1 BFS. Третий: загрузить реальную дорожную сеть – DIMACS USA – с весами, квантованными до $k = 8$. Четвёртый, стретч-цель: вариант Δ -stepping для параллельного контекста. Уже встроено: 32-битный вариант расстояния, корректен при $k \cdot V < 2^{32}$. Именно это позволяет $V \approx 10^9$ влезать в 32 ГБ оперативной памяти. Спасибо – вопросы?”

Вероятные вопросы:

- Почему 0–1 BFS быстрее 0– k BFS при $k = 1$? – один `std::deque` против двух `std::queue`: накладные расходы аллокатора и кэша.
- Как проверяли корректность на запуске с миллиардом вершин? – суммы расстояний совпадают с Дейкстрой побайтово; сумма фиксирует массив без поэлементного сравнения.

- *А параллелизм?* – весь Этап 1 в один поток; Δ -stepping запланирован на Неделю 2.